

COS 135 Individual Project 2 – Process Queue Simulator (Mini OS Scheduler)

Due: Monday 05/04/26 End of the day

What to submit:

- **.c source code.**
- **A two-minute video demonstration** of your code, algorithms, and program execution.
- Your programs must produce similar outputs, as given, if the same inputs are provided.
- Program must compile without errors or warnings (use -Wall switch in gcc compiler).

Comments are required in the following locations (in each C source code):

- At the top of the source code comment your name and a short program description.
- Comment the purpose of each variable.
- Comment major sections of code such as functions, inputs, algorithms, and outputs.

Program Design: Your program is a professional document and must be neat and easy to read. All programs should follow these specifications.

- Comments should be aligned and entered in a consistent fashion
- Blank lines should be added to aid readability
- Code within blocks should be indented
- Comments should not contain spelling mistakes
- Variable names, function names, and other identifiers should be meaningful
- Use loops, arrays, and functions appropriately
- Handle invalid user input gracefully

Write a C program for the following project and submit your source codes (you must submit a .c file **without compilation warnings or errors**). Sample Program inputs/outputs are given below.

Use the -Wall switch in gcc to enable all warnings:

```
$ gcc scheduler.c -o scheduler -Wall
```

Important:

- You are **ONLY ALLOWED** to use C concepts that we have covered in the course.
- Your code **MUST be based on the provided skeleton code**. You are only allowed to complete and modify the given structure (do not rewrite the entire program).
- You **MUST** use dynamic memory allocation (`malloc`, `calloc`, `realloc`, `free`).
- Programs that use advanced libraries or concepts not covered in class **may receive a grade reduction**.

Project Description

A Process Queue Simulator shows how a computer runs multiple tasks one after another. Since a computer can only run one task at a time, other tasks must wait in a queue until it is their turn. For this project, assume a single-CPU system, meaning only one process can run at any moment. We do not consider computers with multiple CPUs running tasks in parallel. In this project, you will build a simple simulator that uses the First-Come, First-Served (FCFS) method. This means the first process added to the queue will be the first one to run.

Each process has:

- A Process ID (PID)
- A Burst Time (how long it takes to run – execution time)

Your program will use structs, pointers, and dynamic memory allocation to store and manage processes. You will also simulate execution over time using delays so you can see each process running step by step.

Your Process Queue Simulator should have the following features:

(a) A data structure to keep the information on processes

- Define a struct to represent a process
- Each process must include:
 - Process ID (PID)
 - Burst Time

(b) Dynamic Queue

- Implement a dynamic array of processes (array of structs)
- Start with an empty queue
- Use `malloc` or `calloc` to allocate memory
- Use `realloc` to expand the queue when adding new processes
- Maintain insertion order (FIFO behavior)

(c) Add Process

- The user should be able to add processes
- Input should include:
 - PID
 - Must be unique. No duplicate PIDs allowed
 - Burst Time
 - New processes must be added to the end of the queue
 - The queue must dynamically grow as needed

(d) Display Queue

- Display all processes in the queue
- Example output:

PID	Burst Time
101	3
102	8
103	5

(e) Execute Scheduler with Time Simulation (FCFS)

- Simulate execution of processes in order (FCFS)
- For each process, display execution progress one time unit at a time
- Use a loop that runs for the duration of the process's burst time
- Use `sleep(1)` to pause execution for 1 second per time unit
- Example output:

```
Executing PID 101
  Time unit 1...
  Time unit 2...
  Time unit 3...
Process 101 completed.
```

- You MUST include:

```
#include <unistd.h>
```

```
sleep(1);
```

- The delay must occur inside the execution loop
- Output must clearly show progress of execution

(f) Memory Management

- Free all dynamically allocated memory before program ends
- Program **SHOULD NOT** have memory leaks

Task Requirements

1. Initialize the Queue

- a. Start with an empty dynamic array
- b. No fixed-size arrays allowed

2. Add Processes

- a. Prompt the user to enter PID and burst time
- b. Store processes dynamically
- c. Expand memory using `realloc`

3. Display Processes

- a. Print all processes in order
- b. Use clear formatting

4. Scheduler Logic (FCFS with Delay)

- a. Execute processes in the order they were added to the queue
- b. For each process:
 - i. Print execution start
 - ii. Loop through burst time
 - iii. Print each time unit
 - iv. Call `sleep(1)` in each iteration (assume 1 time unit = 1 second)
- c. Print completion message

5. User Input and Validation

- a. Validate numeric input
- b. Handle invalid inputs gracefully
- c. Prevent crashes

6. End of Program

- a. Free all allocated memory using `free()`
- b. Exit cleanly

Video Demonstration (~2 minutes):

- a. **Clearly explain your code in your own words.** Your explanation should include:
 - i. Overview of how your program works
 - ii. How dynamic memory is used (`malloc`, `realloc`)
 - iii. Explanation of your struct and functions
 - iv. How your queue and scheduler work
 - v. How `sleep()` is used to simulate execution
- b. Show your program compiling using `-Wall`
- c. Demonstrate live program execution (add processes, display, run scheduler)

Note: The purpose of this video is to demonstrate your understanding of your code, not just to show that it runs. Explain your code clearly without reading from a script. Videos that show code without explanation or demonstrate lack of understanding may receive reduced credit.

Video Recording Instructions (2 minutes)

You can record your video demonstration using 1) Zoom, 2) in-built screen recording tools, or 3) a phone camera. The video must clearly show your code running in the Ubuntu terminal while you explain.

Note: make sure audio is recording properly, especially if you are using the option 2.

Option 1: Zoom (desktop sharing while recording)

- Share your screen and record while demonstrating your terminal session.

Option 2: Built-in Screen Recording Tools

Windows 10/11

Approach 1: Snipping Tool

- Search for Snipping Tool in the Start menu and open it.
- Click the Video Camera icon (Record) in the toolbar.
- Click New and drag to select the screen area to record.
- Adjust microphone and system audio settings, then click Start.
- Click Stop (red square) to end, then save the recording.

Approach 2: Xbox Game Bar

- Press Win + G to open the Xbox Game Bar.
- Click the Record button (or press Win + Alt + R) to start recording.
- Record your terminal session and explanation.
- Press the same keys to stop; the video will be saved in Videos > Captures.

Mac (macOS)

- Press Shift + Command + 5 to open the screen recording toolbar.
- Select Record Entire Screen or Record Selected Portion.
- Click Record, demonstrate your code, and explain.
- Press Stop in the menu bar; the video saves to your desktop by default.

Ubuntu

- Press Ctrl + Alt + Shift + R to start/stop recording (works in Ubuntu with GNOME desktop).
- Video is saved in the Videos folder.

Option 3: Phone Camera

- Place your phone so it clearly captures your computer screen while recording your voice.
- Make sure the screen is readable and text is visible.
- Keep video steady and clear.

Sample program execution:

```
cos135@NRC: ~  
cos135@NRC:~$ gcc /media/sf_COS135_VM_Codes/P2/scheduler.c -o scheduler -Wall  
cos135@NRC:~$ ./scheduler  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 1  
Enter PID: 111  
Enter Burst Time: 3  
Process added successfully.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 1  
Enter PID: 123  
Enter Burst Time: 5  
Process added successfully.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 
```

```
cos135@NRC: ~  
Enter choice: 1  
Enter PID: 123  
Enter Burst Time: 5  
Process added successfully.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 1  
Enter PID: 222  
Enter Burst Time: 6  
Process added successfully.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 2  
  
PID    Burst Time  
111     3  
123     5  
222     6  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 
```

```
cos135@NRC: ~  
3. Run Scheduler  
4. Exit  
Enter choice: 3  
  
Executing PID 111  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
Process 111 completed.  
  
Executing PID 123  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
  Time unit 4...  
  Time unit 5...  
Process 123 completed.  
  
Executing PID 222  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
  Time unit 4...  
  Time unit 5...  
  Time unit 6...  
Process 222 completed.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 
```

```
cos135@NRC: ~  
Enter choice: 3  
  
Executing PID 111  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
Process 111 completed.  
  
Executing PID 123  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
  Time unit 4...  
  Time unit 5...  
Process 123 completed.  
  
Executing PID 222  
  Time unit 1...  
  Time unit 2...  
  Time unit 3...  
  Time unit 4...  
  Time unit 5...  
  Time unit 6...  
Process 222 completed.  
  
--- Process Scheduler Menu ---  
1. Add Process  
2. Display Queue  
3. Run Scheduler  
4. Exit  
Enter choice: 4  
Exiting program...  
cos135@NRC: ~$ 
```

Errors or Invalid inputs:

```
cos135@NRC:~$ ./scheduler

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 1
Enter PID: 111
Enter Burst Time: 2
Process added successfully.

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 1
Enter PID: 111
Error: Duplicate PID not allowed.

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 
```

```
cos135@NRC:~$ ./scheduler

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 2
Queue is empty.

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 3
No processes to execute.

--- Process Scheduler Menu ---
1. Add Process
2. Display Queue
3. Run Scheduler
4. Exit
Enter choice: 4
Exiting program...
cos135@NRC:~$ 
```


Grading Rubric (80 pts):

Process Structure (6 pts)

- a. Struct correctly defined (2 pts)
- b. PID and Burst Time stored and used correctly (4 pts)

Dynamic Queue & Memory (20 pts)

- a. Dynamic array implemented using structs (4 pts)
- b. Uses malloc or calloc correctly (4 pts)
- c. Uses realloc correctly to expand queue (8 pts)
- d. No fixed-size arrays used (2 pts)
- e. Memory freed properly (no leaks) (2 pts)

Add Process Functionality (14 pts)

- a. User can input PID and burst time correctly (4 pts)
- b. Processes added to queue in correct order (FIFO) (6 pts)
- c. Prevents duplicate PIDs (4 pts)

Display Queue (6 pts)

- a. Output formatting is clear and readable (3 pts)
- b. All processes displayed correctly (3 pts)

Scheduler Execution with Delay (16 pts)

- a. Processes executed in correct FCFS order (6 pts)
- b. Execution loop correctly uses burst time (4 pts)
- c. Uses sleep(1) correctly inside loop (2 pts)
- d. Clear step-by-step execution output (4 pts)

Input Validation & Error Handling (3 pts)

- a. Handles invalid inputs (non-numeric, out-of-range) (2 pts)
- b. Program does not crash (1 pt)

Code Structure & Readability (8 pts)

- a. Proper use of functions (2 pts)
- b. Meaningful variable names (2 pts)
- c. Code is well-indented and readable (2 pts)
- d. Required comments included (2 pts)

Video Demonstration (7 pts)

- a. Clearly explains code and logic (5 pts)
- b. Demonstrates program running and understanding (2 pts)